

读享 V1.0 软件

前 30 页

著作权人:XX 有限公司

生成日期:2026-06-15

摘选总行数:1500

文件清单:

- person/notfresh/readingshare/MainActivity.java
- person/notfresh/readingshare/ShareUtil.java
- person/notfresh/readingshare/ui/home/HomeFragment.java

```
1  package person.notfresh.readingshare;
2
3  import android.annotation.SuppressLint;
4  import android.os.Bundle;
5  import android.view.LayoutInflater;
6  import android.view.View;
7  import android.view.Menu;
8  import android.view.MenuItem;
9  import android.content.Intent;
10 import android.net.Uri;
11
12 import java.util.ArrayList;
13 import java.util.List;
14 import java.util.Map;
15
16 import android.util.Log;
17 import android.content.pm.PackageManager;
18 import android.content.pm.ResolveInfo;
19
20 import com.google.android.flexbox.FlexboxLayout;
21 import com.google.android.material.snackbar.Snackbar;
22 import com.google.android.material.navigation.NavigationView;
23
24 import androidx.navigation.NavController;
25 import androidx.navigation.NavDestination;
26 import androidx.navigation.Navigation;
27 import androidx.navigation.ui.AppBarConfiguration;
28 import androidx.navigation.ui.NavigationUI;
29 import androidx.drawerlayout.widget.DrawerLayout;
30 import androidx.appcompat.app.AppCompatActivity;
31 import androidx.fragment.app.Fragment;
32
33 import person.notfresh.readingshare.databinding.ActivityMainBinding;
34 import person.notfresh.readingshare.db.LinkDao;
35 import person.notfresh.readingshare.model.LinkItem;
36 import person.notfresh.readingshare.ui.home.HomeFragment;
37 import android.content.ClipboardManager;
38 import android.content.ClipData;
39 import android.app.AlertDialog;
40 import android.widget.EditText;
41 import android.text.TextUtils;
42 import java.io.BufferedReader;
43 import java.io.InputStreamReader;
44 import java.net.URL;
45 import java.nio.charset.StandardCharsets;
46 import android.os.Handler;
47 import android.os.Looper;
48 import java.util.concurrent.ExecutorService;
49 import java.util.concurrent.Executors;
50 import java.util.regex.Matcher;
```

```
100         DrawerLayout drawer = binding.drawerLayout;
```

```

101 NavigationView navigationView = binding.navView;
102
103 // ■■■■■■■■
104 navController = Navigation.findNavController(this, R.id.nav_host_fra
105
106 // ■■■■■■■■DocumentViewerActivity■■■■■■■
107 handleNavigation(getIntent());
108
109 // ■■■■ AppBarConfiguration
110 // ■■■nav_tags■■■■■■■■■■nav_home
111 mAppBarConfiguration = new AppBarConfiguration.Builder(
112     R.id.nav_home, R.id.nav_slideshow, R.id.nav_rss, R.id.nav_ar
113     .setOpenableLayout(drawer)
114     .build();
115
116 NavigationUI.setupActionBarWithNavController(this, navController, mA
117 NavigationUI.setupWithNavController(navigationView, navController);
118
119 // ■■■■■■■■■■
120 View headerView = navigationView.getHeaderView(0);
121 headerView.setOnClickListener(v -> {
122     drawer.closeDrawer(GravityCompat.START);
123     new Handler().postDelayed(() -> {
124         Intent intent = new Intent(this, UserProfileActivity.class);
125         startActivity(intent);
126     }, 250);
127 });
128
129 // ■■■■■■
130 handleIntent(getIntent());
131
132 // ■■■■■■■■■■
133 navigationView.setNavigationItemSelectedListener(item -> {
134     int id = item.getItemId();
135     drawer.closeDrawer(GravityCompat.START);
136
137     new Handler().postDelayed(() -> {
138         try {
139             if (id == R.id.nav_home) {
140                 navController.navigate(R.id.nav_home);
141             } else if (id == R.id.nav_rss) {
142                 navController.navigate(R.id.nav_rss);
143             } else if (id == R.id.nav_slideshow) {
144                 navController.navigate(R.id.nav_slideshow);
145             } else if (id == R.id.nav_archive) {
146                 navController.navigate(R.id.nav_archive);
147             } else if (id == R.id.nav_documents) {
148                 navController.navigate(R.id.nav_documents);
149             } else if (id == R.id.nav_subject) {
150                 navController.navigate(R.id.nav_subject);

```

```
@Override
```

第6页 / 共31页

第7页 / 共31页

第8页 / 共31页


```
400         Log.d("Clipboard", "App is not in focus or foreground");
```

第 10 页 / 共 31 页

```
497  
498 /** ██████████ */
```


600

第 14 页 / 共 31 页

```
651         // clipboard.setPrimaryClip(ClipData.newPlainText("", ""));
652     });
653
654     dialog.show();
655
656     // ■■■■■■■■■■
657     fetchTitleFromUrl(url, titleInput);
658 }
659
660 private void fetchTitleFromUrl(String url, EditText titleInput) {
661     ExecutorService executor = Executors.newSingleThreadExecutor();
662     Handler handler = new Handler(Looper.getMainLooper());
663
664     executor.execute(() -> {
665
666         String title = "";
667         try {
668             if (url.contains("weixin.qq.com")) {
669                 title = CrawlUtil.getWeixinArticleTitle(url);
670             } else {
671                 title = CrawlUtil.fetchTitleCommon(url);
672             }
673         } catch (Exception e) {
674             Log.e("FetchTitle", "Error fetching title", e);
675         }
676
677         final String finalTitle = title;
678         handler.post(() -> {
679             if (!TextUtils.isEmpty(finalTitle)) {
680                 titleInput.setText(finalTitle);
681             }
682         });
683     });
684 }
685
686 @SuppressWarnings("ResourceType")
687 private void updateNavHeader() {
688     try {
689         NavigationView navigationView = binding.navView;
690         View headerView = navigationView.getHeaderView(0);
691         ImageView profileImage = headerView.findViewById(R.id.nav_header_ima
692         TextView usernameText = headerView.findViewById(R.id.nav_header_user
693         TextView emailText = headerView.findViewById(R.id.nav_header_email);
694
695         SharedPreferences prefs = getSharedPreferences("UserProfile", MODE_P
696         String username = prefs.getString("username", getString(R.string.nav
697         String email = prefs.getString("email", getString(R.string.nav_heade
698         String imageUri = prefs.getString("profile_image", "");
699
700         usernameText.setText(username);
```

第 16 页 / 共 31 页

读享 V1.0 软件 - 源程序代码(前 30 页)

文件:person/notfresh/readingshare/ShareUtil.java 起始行:1

```
1 // MISSING: person/notfresh/readingshare/ShareUtil.java
```

```
1  package person.notfresh.readingshare.ui.home;
2
3  import android.app.Activity;
4  import android.content.Intent;
5  import android.graphics.Bitmap;
6  import android.net.Uri;
7  import android.os.Bundle;
8  import android.text.Editable;
9  import android.text.TextWatcher;
10 import android.util.Log;
11 import android.view.LayoutInflater;
12 import android.view.Menu;
13 import android.view.MenuInflater;
14 import android.view.MenuItem;
15 import android.view.View;
16 import android.view.ViewGroup;
17 import android.widget.EditText;
18 import android.widget.PopupWindow;
19 import android.widget.Toast;
20
21 import androidx.annotation.NonNull;
22 import androidx.annotation.Nullable;
23 import androidx.fragment.app.Fragment;
24 import androidx.navigation.Navigation;
25 import androidx.recyclerview.widget.ItemTouchHelper;
26 import androidx.recyclerview.widget.LinearLayoutManager;
27 import androidx.recyclerview.widget.RecyclerView;
28
29 import com.google.android.flexbox.FlexboxLayoutManager;
30 import com.google.android.flexbox.FlexDirection;
31 import com.google.android.flexbox.FlexWrap;
32
33 import com.google.android.material.snackbar.Snackbar;
34
35 import person.notfresh.readingshare.R;
36 import person.notfresh.readingshare.adapter.LinksAdapter;
37 import person.notfresh.readingshare.adapter.SearchHistoryAdapter;
38 import person.notfresh.readingshare.adapter.TagsAdapter;
39 import person.notfresh.readingshare.databinding.FragmentHomeBinding;
40 import person.notfresh.readingshare.db.LinkDao;
41 import person.notfresh.readingshare.db.SearchHistoryManager;
42 import person.notfresh.readingshare.db.SubjectDao;
43 import person.notfresh.readingshare.model.LinkItem;
44 import person.notfresh.readingshare.model.SearchHistoryItem;
45 import person.notfresh.readingshare.core.model.SubjectItem;
46 import person.notfresh.readingshare.core.model.SubjectUtil;
47 import person.notfresh.readingshare.ui.subject.SelectSubjectDialog;
48 import person.notfresh.readingshare.ClickStatisticsActivity;
49 import person.notfresh.readingshare.util.ShareUtil;
50 import person.notfresh.readingshare.util.ImageUtil;
```

100

```

101     private boolean isSelectionMode = false;
102     private MenuItem shareMenuItem;
103     private MenuItem closeSelectionMenuItem;
104     private MenuItem enterSelectionMenuItem;
105     private MenuItem selectAllMenuItem; // ■■■■■■■■TagsFragment■■■
106     private MenuItem addTagMenuItem; // ■■■■■■■■■■TagsFragment■■■
107     private MenuItem sortMenuItem; // ■■■■■■■■TagsFragment■■■
108     private MenuItem exitSortMenuItem; // ■■■■■■■■■■TagsFragment■■■
109     private EditText searchEditText;
110
111     // ===== ■■■■■■ =====
112     private static final long SEARCH_DEBOUNCE_MS = 300L;
113     private static final long SEARCH_BLUR_HIDE_DELAY_MS = 200L;
114     private static final int SEARCH_HISTORY_POPUP_MAX_DP = 300;
115     private static final int SEARCH_DEFAULT_END_PADDING_DP = 8;
116     private static final int SEARCH_CLEAR_END_PADDING_DP = 36;
117
118     private SearchHistoryManager searchHistoryManager;
119     private List<SearchHistoryItem> historyCache = new ArrayList<>();
120     private PopupWindow historyPopup;
121     private SearchHistoryAdapter historyAdapter;
122     private final Runnable debounceSearchRunnable = new Runnable() {
123         @Override public void run() { performSearch(searchEditText.getText().toS
124     };
125     private final Runnable hidePopupRunnable = new Runnable() {
126         @Override public void run() { dismissHistoryPopup(); }
127     };
128     private ImageButton searchClearButton;
129
130     // ■■■■■■■■■■■■■■■■■■■■■■
131     private LinkItem pendingIconItem;
132     private String pendingIconUrl;
133
134     // ===== ■■■■■■■■■■■■■■TagsFragment■■■=====
135     private static final String PREF_NAME = "TagsPreferences";
136     private static final String KEY_SELECTED_TAGS = "selectedTags";
137     private static final String KEY_NO_TAG_SELECTED = "noTagSelected";
138     private static final String NO_TAG = "NO_TAG"; // ■■■■"■■■"■■■
139     private static final String PREF_HIGHLIGHTED_TAGS = "highlighted_tags";
140     private static final int FIXED_TAGS_COUNT = 10; // ■■■■■■■■■■
141     private static final int COLLAPSED_HEIGHT_DP = 120; // ■■■■
142     private static final int SORT_MODE_MAX_HEIGHT_DP = 400; // ■■■■■■■■■■■■■■■dp■
143
144     // ■■■■■■■■■■
145     private static final String KEY_SHUFFLE_MODE = "shuffle_mode";
146     private static final long LINK_ADD_INTERVAL_MS = 5000; // 5■■■■■■■■■■■■■■■■■■■■
147
148     private RecyclerView tagsRecyclerView;
149     private RecyclerView tagsRecyclerViewCollapsed; // ■■■■■■
150     private TagsAdapter tagsAdapter;

```

[illegible]

[illegible]

```
251         return true;
252     } else if (id == R.id.action_share_text) {
253         shareAsText();
254         return true;
255     } else if (id == R.id.action_share_json) {
256         shareAsFile(true); // true ■■ JSON
257         return true;
258     } else if (id == R.id.action_share_csv) {
259         shareAsFile(false); // false ■■ CSV
260         return true;
261     } else if (id == R.id.action_add_to_subject) {
262         addToSubject();
263         return true;
264     } else if (id == R.id.action_statistics) {
265         // ■■■■■■■■
266         Navigation.findNavController(requireView())
267             .navigate(R.id.action_nav_home_to_nav_statistics);
268         return true;
269     } else if (id == R.id.action_statistics_read) {
270         // ■■■■■■■■■■
271         Intent intent = new Intent(getActivity(), ClickStatisticsActivity.cl
272             startActivity(intent);
273         return true;
274     } else if (id == R.id.action_add_tag) {
275         // ■■■■■■■■TagsFragment■■■
276         showAddTagDialog();
277         return true;
278     } else if (id == R.id.action_sort_tags) {
279         // ■■■■■■■■■■TagsFragment■■■
280         toggleSortMode();
281         return true;
282     } else if (id == R.id.action_exit_sort) {
283         // ■■■■■■■■■■TagsFragment■■■
284         toggleSortMode();
285         return true;
286     } else if (id == R.id.action_select_all) {
287         // ■■■■TagsFragment■■■
288         selectAllItems();
289         return true;
290     } else if (id == R.id.action_shuffle) {
291         if (isShuffleMode) {
292             // ■■■■■■■■■■■■■■
293             reshuffleLinks();
294         } else {
295             // ■■■■■■
296             toggleShuffleMode();
297         }
298         return true;
299     } else if (id == R.id.action_exit_shuffle) {
300         // ■■■■■■
```

```

301         toggleShuffleMode();
302         return true;
303     }
304     return super.onOptionsItemSelected(item);
305 }
306
307 @Override
308 public View onCreateView(@NonNull LayoutInflater inflater,
309                          ViewGroup container, Bundle savedInstanceState) {
310     Log.d(TAG, "onCreateView: start");
311     try {
312         binding = FragmentHomeBinding.inflate(inflater, container, false);
313         View root = binding.getRoot();
314         Log.d(TAG, "onCreateView: layout loaded successfully");
315
316         // ■■■■■■■■■■
317         String selectedDate = null;
318         if (getArguments() != null) {
319             selectedDate = getArguments().getString("selected_date");
320         }
321
322         Log.d(TAG, "onCreateView: initializing LinkDao");
323         linkDao = new LinkDao(requireContext());
324         linkDao.open();
325         Log.d(TAG, "onCreateView: LinkDao opened successfully");
326
327         Log.d(TAG, "onCreateView: initializing RecyclerView and Adapter");
328         RecyclerView recyclerView = binding.recyclerView;
329         if (recyclerView == null) {
330             Log.e(TAG, "onCreateView: recyclerView is null!");
331             throw new IllegalStateException("recyclerView■null");
332         }
333
334         adapter = new LinksAdapter(requireContext());
335         adapter.setOnLinkActionListener(this);
336         recyclerView.setAdapter(adapter);
337         recyclerView.setLayoutManager(new LinearLayoutManager(requireContext));
338         Log.d(TAG, "onCreateView: RecyclerView setup completed");
339
340         // ■■■■■■■■■■
341         adapter.enableSwipeActions(recyclerView);
342
343         // ■■ RecyclerView ■■■■■■
344         recyclerView.setOnTouchListener((v, event) -> {
345             if (searchEditText != null) {
346                 searchEditText.clearFocus(); // ■■■■■■■■■■
347             }
348             return false;
349         });
350

```



```

351         // ■■■■■■
352         Log.d(TAG, "onCreateView: initializing search box");
353         searchText = binding.searchEditText;
354         if (searchEditText != null) {
355             // ■■■■■■■■
356             searchHistoryManager = new SearchHistoryManager(requireContext())
357             refreshHistoryCache();
358
359             searchText.addTextChangedListener(new TextWatcher() {
360                 @Override public void beforeTextChanged(CharSequence s, int
361                 @Override public void onTextChanged(CharSequence s, int star
362                 @Override
363                 public void afterTextChanged(Editable s) {
364                     // ■■■■ x ■■■■■■■■■■ 300ms ■■■■
365                     updateInputChrome();
366                     // ■■■■■■■■■■ + ■■■■
367                     searchText.removeCallbacks(debounceSearchRunnable);
368                     searchText.postDelayed(debounceSearchRunnable, SEARC
369                 }
370             });
371
372             // focus: ■■■■ updateInputChrome ■■■■■■■■■■
373             searchText.setOnFocusChangeListener((v, hasFocus) -> {
374                 if (hasFocus) {
375                     updateInputChrome();
376                 } else {
377                     // 200ms ■■■■■■■■■■■■■■■■
378                     searchText.removeCallbacks(hidePopupRunnable);
379                     searchText.postDelayed(hidePopupRunnable, SEARCH_BLU
380                 }
381             });
382
383             // x ■■■■■■■■■■ + ■■■■ + ■ updateInputChrome ■■■■■■■■■■
384             searchClearButton = binding.searchClearButton;
385             if (searchClearButton != null) {
386                 searchClearButton.setOnClickListener(v -> {
387                     searchText.removeCallbacks(debounceSearchRunnable);
388                     searchText.setText("");
389                     // ■■■■■■setText ■■■■ TextWatcher
390                     updateInputChrome();
391                 });
392             }
393         } else {
394             Log.w(TAG, "onCreateView: searchText is null");
395         }
396
397         // ===== ■■■■■■■■UI■■■■2■■■■=====
398         Log.d(TAG, "onCreateView: initializing tag management UI");
399         try {
400             initTagRecyclerView(root);

```

450

```

496  /** ██████████ EditText ████████████████████ */
497  private void showHistoryPopup() {
498      if (searchEditText == null || searchEditText.getWindowToken() == null) r
499      if (historyCache.isEmpty()) return;    // ██████████
500      refreshHistoryCache();                // ████████████████████████████████

```

```
501         if (historyPopup == null) {
502             buildHistoryPopup();
503         }
504         if (historyPopup != null && !historyPopup.isShowing()) {
505             historyPopup.showAsDropDown(searchEditText, 0, 0);
506         }
507     }
508
509     private void dismissHistoryPopup() {
510         if (historyPopup != null && historyPopup.isShowing()) {
511             historyPopup.dismiss();
512         }
513     }
514
515     /** ■■■ PopupWindow■■■■■■■■■■ */
516     private void buildHistoryPopup() {
517         Context ctx = requireContext();
518         historyAdapter = new SearchHistoryAdapter();
519         historyAdapter.setOnItemClickListener(new SearchHistoryAdapter.OnItemCli
520             @Override public void onItemClick(SearchHistoryItem item) {
521                 String kw = item.getText();
522                 searchEditText.removeCallbacks(hidePopupRunnable);
523                 searchEditText.setText(kw);
524                 searchEditText.setSelection(kw.length());
525                 searchEditText.removeCallbacks(debounceSearchRunnable);
526                 dismissHistoryPopup();
527                 // ■■■■■■■■■■setText ■■■■ TextWatcher
528                 performSearch(kw);
529                 updateInputChrome();
530             }
531             @Override public void onPinClick(SearchHistoryItem item) {
532                 searchHistoryManager.togglePinKeyword(item.getText());
533                 refreshHistoryCache();
534                 updateInputChrome();
535             }
536             @Override public void onDeleteClick(SearchHistoryItem item) {
537                 searchHistoryManager.deleteKeyword(item.getText());
538                 refreshHistoryCache();
539                 updateInputChrome();
540             }
541         });
542         historyAdapter.setItems(historyCache);
543
544         RecyclerView rv = new RecyclerView(ctx);
545         rv.setLayoutManager(new LinearLayoutManager(ctx));
546         rv.setAdapter(historyAdapter);
547         rv.setBackgroundResource(R.drawable.search_history_popup_bg);
548         rv.setElevation(8f);
549
550         int width = searchEditText.getWidth();
```

```

551 int maxHeight = (int) (SEARCH_HISTORY_POPUP_MAX_DP * ctx.getResources().
552 historyPopup = new PopupWindow(rv, width, ViewGroup.LayoutParams.WRAP_CO
553 historyPopup.setBackgroundDrawable(new android.graphics.drawable.ColorDr
554 historyPopup.setOutsideTouchable(false);
555 historyPopup.setFocusable(false); // ██████████ EditText ███
556 historyPopup.setHeight(maxHeight);
557
558 // ██████████ pending hide spec 6.6 ██████
559 rv.setOnTouchListener((v, ev) -> {
560     searchEditText.removeCallbacks(hidePopupRunnable);
561     return false;
562 });
563 }
564
565 @Override
566 public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
567     super.onViewCreated(view, savedInstanceState);
568     Log.d(TAG, "onViewCreated: start");
569
570     // █onViewCreated████████████████████RecyclerView██████████
571     // ████████████████████████████████████████
572     if (linkDao != null && adapter != null) {
573         Log.d(TAG, "onViewCreated: loading all links directly");
574         List<LinkItem> pinnedLinks = linkDao.getPinnedLinks();
575         Map<String, List<LinkItem>> groupedLinks = linkDao.getLinksGroupByDa
576
577         // ████████
578         int totalLinks = pinnedLinks.size();
579         for (List<LinkItem> links : groupedLinks.values()) {
580             totalLinks += links.size();
581         }
582
583         Log.d(TAG, "onViewCreated: loaded links - pinned=" + pinnedLinks.siz
584             + ", groups=" + groupedLinks.size() + ", total links=" + tot
585
586         // ██████████adapter
587         adapter.setPinnedLinks(pinnedLinks);
588         adapter.setGroupedLinks(groupedLinks);
589         Log.d(TAG, "onViewCreated: data set to adapter, itemCount=" + adapte
590
591         // ███RecyclerView███
592         RecyclerView recyclerView = binding.recyclerView;
593         if (recyclerView != null) {
594             Log.d(TAG, "onViewCreated: RecyclerView state - visibility=" + r
595                 + " (0=VISIBLE), width=" + recyclerView.getWidth()
596                 + ", height=" + recyclerView.getHeight()
597                 + ", isShown=" + recyclerView.isShown()
598                 + ", hasFixedSize=" + recyclerView.hasFixedSize());
599
600             // ███RecyclerView███

```

第 30 页 / 共 31 页

```

651     private void initTagRecyclerView(View root) {
652         Log.d(TAG, "initTagRecyclerView: start");
653         try {
654             if (root == null) {
655                 Log.e(TAG, "initTagRecyclerView: root is null!");
656                 return;
657             }
658
659             // ■■■■■■■■■■2■■■■■
660             tagsContainer = root.findViewById(R.id.tags_container);
661             if (tagsContainer != null) {
662                 tagsContainer.setVisibility(View.VISIBLE); // ■■2■■■■■■■■■■
663                 Log.d(TAG, "initTagRecyclerView: tags container displayed");
664             } else {
665                 Log.w(TAG, "initTagRecyclerView: tagsContainer is null, may not
666             }
667
668             // ■■■■■■■■ RecyclerView
669             Log.d(TAG, "initTagRecyclerView: initializing fixed tags RecyclerView");
670             tagsRecyclerView = root.findViewById(R.id.recycler_tags_fixed);
671             if (tagsRecyclerView != null) {
672                 try {
673                     FlexboxLayoutManager layoutManager = new FlexboxLayoutManager
674                     layoutManager.setFlexDirection(FlexDirection.ROW);
675                     layoutManager.setFlexWrap(FlexWrap.WRAP);
676                     tagsRecyclerView.setLayoutManager(layoutManager);
677
678                     tagsAdapter = new TagsAdapter();
679                     tagsAdapter.setOnTagClickListener((position, tag) -> {
680                         if (!isSortMode) {
681                             handleTagClick(tag.getName());
682                         }
683                     });
684                     tagsAdapter.setOnTagLongClickListener((position, tag) -> {
685                         if (!isSortMode) {
686                             showTagOptionsDialog(tag.getName());
687                         }
688                     });
689                     tagsRecyclerView.setAdapter(tagsAdapter);
690                     Log.d(TAG, "initTagRecyclerView: fixed tags RecyclerView ini
691                 } catch (Exception e) {
692                     Log.e(TAG, "initTagRecyclerView: fixed tags RecyclerView ini
693                 }
694             } else {
695                 Log.w(TAG, "initTagRecyclerView: recycler_tags_fixed is null");
696             }
697
698             // ■■■■■■■■ RecyclerView
699             Log.d(TAG, "initTagRecyclerView: initializing collapsed tags Recycle
700             tagsRecyclerViewCollapsed = root.findViewById(R.id.recycler_tags_col

```